BÀI THỰC HÀNH XỬ LÝ TÍN HIỆU SỐ

Trần Duy Tiến - 2413488
Võ Chí Cương - 2410420
15/09/2026

Tổng Quan

Báo cáo này trình bày kết quả của Bài thực hành số 1&2 (Lab 1&2), tập trung vào việc ôn tập các kiến thức về tín hiệu và hệ thống rời rạc. Nội dung bao gồm việc áp dụng phần mềm Scilab để giải quyết các bài tập thực hành liên quan đến việc định nghĩa, hiển thị và thao tác trên các tín hiệu cơ bản.

1 Giới Thiệu

Bài thực hành cung cấp cái nhìn trực quan về các tín hiệu rời rạc cơ bản bao gồm tín hiệu xung đơn vị (unit sample sequence), tín hiệu bước nhảy (unit step signal) và tín hiệu hàm dốc (unit ramp signal). Báo cáo sẽ trình bày chi tiết việc sử dụng Scilab để mô phỏng các thuộc tính của tín hiệu như thành phần chẵn, lẻ và thực hiện các phép biến đổi cơ bản gồm dịch thời gian (delay, advance), đảo thời gian (folding), cộng, nhân và thay đổi biên độ tín hiệu.

2 Mục Tiêu

- Ôn tập lại các kiến thức bài giảng về tín hiệu và hệ thống rời rạc.
- Thực hiện các bài tập thực hành bổ sung trên phần mềm Scilab.
- Chụp ảnh màn hình cho từng hàm hoặc nhóm hàm đã thực hiện để làm minh chứng tự làm và tổng hợp vào một tệp báo cáo duy nhất.

3 Phương Pháp

- **Công cụ phần mềm:** Sử dụng môi trường lập trình Scilab để viết mã lệnh, khảo sát các hàm cơ bản (như `plot2d3`, `bool2s`, `subplot`) và vẽ các đồ thị tín hiệu trên cùng một cửa sổ hiển thị.

- **Cơ sở lý thuyết:** Áp dụng các định nghĩa toán học của tín hiệu rời rạc cơ bản ($\delta(n)$, $u(n)$, $u_r(n)$). Sử dụng các công thức tính toán thành phần chẵn $x_e(n) = \frac{1}{2}[x(n) + x(-n)]$, thành phần lẻ $x_o(n) = \frac{1}{2}[x(n) - x(-n)]$ và các phép biến đổi tín hiệu cơ bản như $x(n-k)$, $x(n+k)$, $x(-n)$ để lập trình mô phỏng.

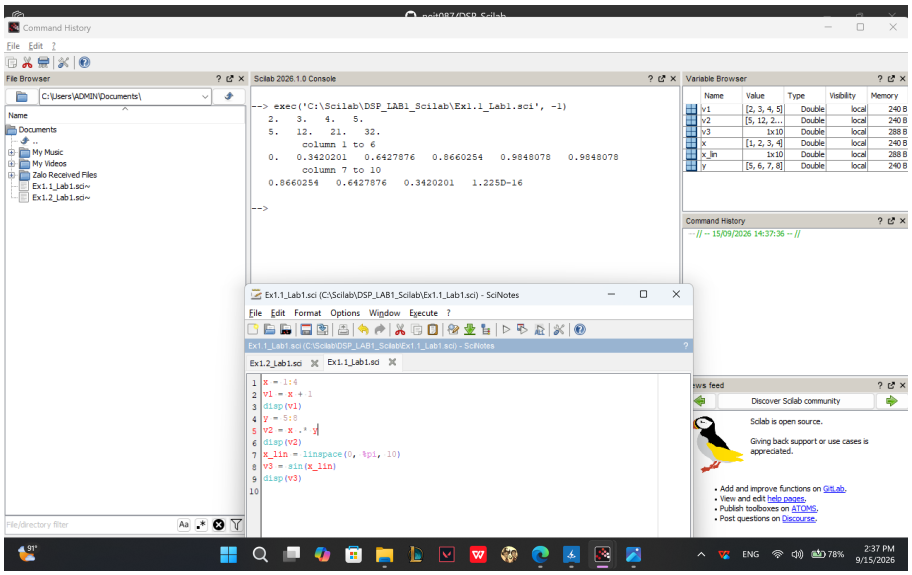
4 Nội Dung Thực Hành

4.1 LAB 1:

4.1.1 Bài 1.1: Thao tác với Vector và Ma trận

Sử dụng các toán tử trong Scilab để tạo các vector theo yêu cầu:

```
x = 1:4
v1 = x + 1
disp(v1)
y = 5:8
v2 = x .* y
disp(v2)
x_lin = linspace(0, %pi, 10)
v3 = sin(x_lin)
disp(v3)
```



Hình 1: Kết quả thực thi mã lệnh bài 1.1 trên Scilab Console

4.1.2 Bài 1.2: Xử lý Tín hiệu Tương tự - Số

Tính toán lý thuyết:

- Analog signal:

$$x_a(t) = 3\sin(100\pi t)$$

- Discrete-time signal sau khi lấy mẫu ($F_s = 300$):

$$x(n) = 3\sin\left(100\pi\frac{n}{300}\right) = 3\sin\left(\frac{\pi}{3}n\right)$$

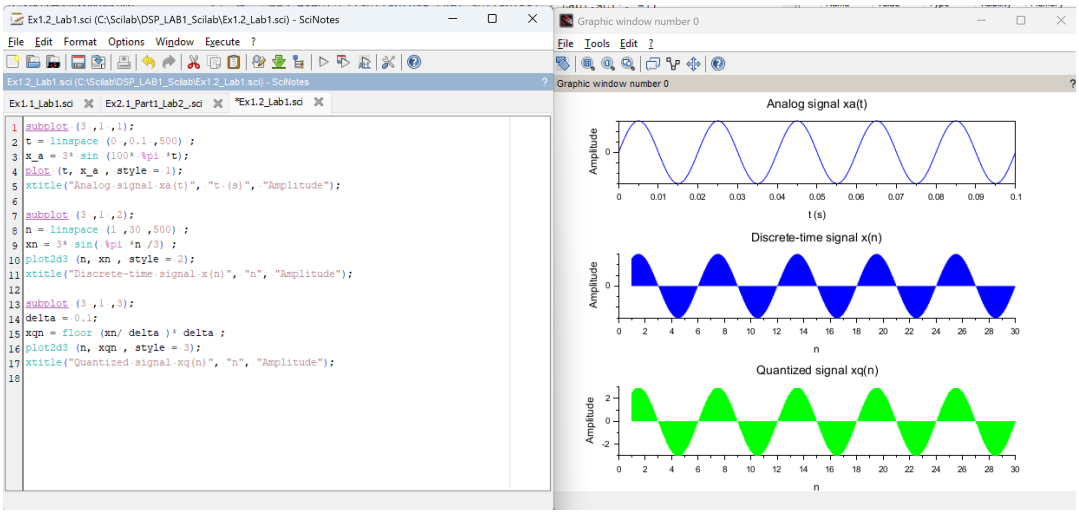
- Xét tính tuần hoàn: Tần số góc $\omega = \frac{\pi}{3}$, suy ra $f = \frac{\omega}{2\pi} = \frac{1}{6}$. Vì f là số hữu tỉ nên $x(n)$ là tín hiệu tuần hoàn với tần số $f = 1/6$ (chu kỳ/mẫu) và chu kỳ $N = 6$ mẫu.

Đồ thị minh chứng: Lượng tử hóa tín hiệu với $\Delta = 0.1$ bằng phương pháp cắt cụt. Cả ba tín hiệu bao gồm analog signal $x_a(t)$, discrete-time signal $x(n)$ và quantized signal $x_q(n)$ đều được lập trình để hiển thị chung trên một cửa sổ duy nhất.

```
subplot (3 ,1 ,1);
t = linspace (0 ,0.1 ,500) ;
x_a = 3* sin (100* %pi *t);
plot (t, x_a , style = 1);
xtitle("Analog signal xa(t)", "t (s)", "Amplitude");

subplot (3 ,1 ,2);
n = linspace (1 ,30 ,500) ;
xn = 3* sin( %pi *n /3) ;
plot2d3 (n, xn , style = 2);
xtitle("Discrete-time signal x(n)", "n", "Amplitude");

subplot (3 ,1 ,3);
delta = 0.1;
xqn = floor (xn/ delta )* delta ;
plot2d3 (n, xqn , style = 3);
xtitle("Quantized signal xq(n)", "n", "Amplitude");
```



Hình 2: Đồ thị mô phỏng analog signal, discrete-time signal và quantized signal

4.2 LAB 2:

4.2.1 Bài 1:

Hãy tìm hiểu một số hàm dưới đây và trình bày về chức năng của chúng cũng như cách sử dụng.

Functions	Description
`plot2d3(...)`	Hàm chuyên dụng để vẽ đồ thị tín hiệu rời rạc dạng rễ cây (stem plot), hiển thị các vạch thẳng đứng nối từ trục hoành đến giá trị biên độ. **Ví dụ:** `plot2d3(n, x);`
`min(...)`	Trả về phần tử có giá trị nhỏ nhất được tìm thấy trong một mảng, vector hoặc ma trận dữ liệu. **Ví dụ:** `gt_nho_nhat = min(x);`
`max(...)`	Trả về phần tử có giá trị lớn nhất được tìm thấy trong một mảng, vector hoặc ma trận dữ liệu. **Ví dụ:** `gt_lon_nhat = max(x);`
`subplot(...)`	Hỗ trợ phân chia một cửa sổ đồ họa thành một lưới ma trận, cho phép hiển thị nhiều đồ thị con độc lập. **Ví dụ:** `subplot(2, 2, 1);` chia lưới 2x2 và đặt đồ thị vào ô số 1.
`title(...)`	Thêm chuỗi văn bản làm tiêu đề chính, hiển thị ở phía trên cùng của đồ thị hiện tại. **Ví dụ:** `title("Tin hieu buoc nhay u(n)");`
`xlabel(...)`	Gắn nhãn mô tả ý nghĩa hoặc đơn vị cho trục hoành (trục X) của đồ thị. **Ví dụ:** `xlabel("Thoi gian roi rac n");`
`ylabel(...)`	Gắn nhãn mô tả ý nghĩa hoặc đơn vị cho trục tung (trục Y) của đồ thị. **Ví dụ:** `ylabel("Bien do (Amplitude)");`
`bool2s(...)`	Chuyển đổi một mảng logic (gồm các giá trị Đúng/Sai) thành một mảng số học (giá trị 1 và 0). Hàm này cực kỳ hữu ích để khởi tạo các tín hiệu cơ sở. **Ví dụ:** `u = bool2s(n >= 0);`
`deff(...)`	Cho phép định nghĩa nhanh một hàm tính toán trực tiếp thông qua chuỗi ký tự (string) mà không cần phải tạo một tệp tin `.sci` riêng biệt. **Ví dụ:** `deff('[y] = ham_mu(x)', 'y = x^2');`

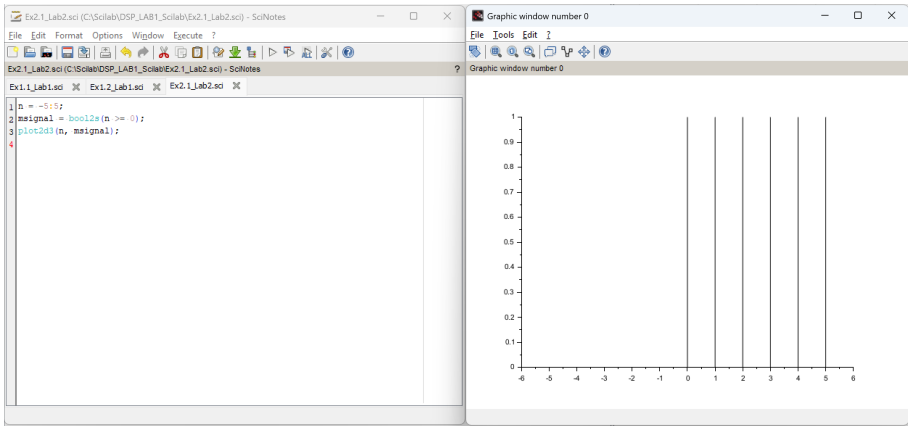
4.2.2 Bài 2:

Yêu cầu: Khảo sát mã lệnh tạo tín hiệu bước nhảy $u(n)$ và giải thích kết quả.

Giải thích: Trong Scilab, biểu thức `n >= 0` tạo ra một mảng logic có giá trị Đúng (True) khi $n \geq 0$ và Sai (False) khi $n < 0$. Khi mảng này đi qua hàm `bool2s()`, các giá trị logic được chuyển đổi tương ứng thành giá trị số học 1 và 0. Kết quả thu được chính là tín hiệu bước nhảy $u(n)$ có biên độ bằng 1 với mọi $n \geq 0$ và bằng 0 với phần còn lại.

```
1 n = -5:5;
```

```
msignal = bool2s(n >= 0);
plot2d3(n, msignal);
```



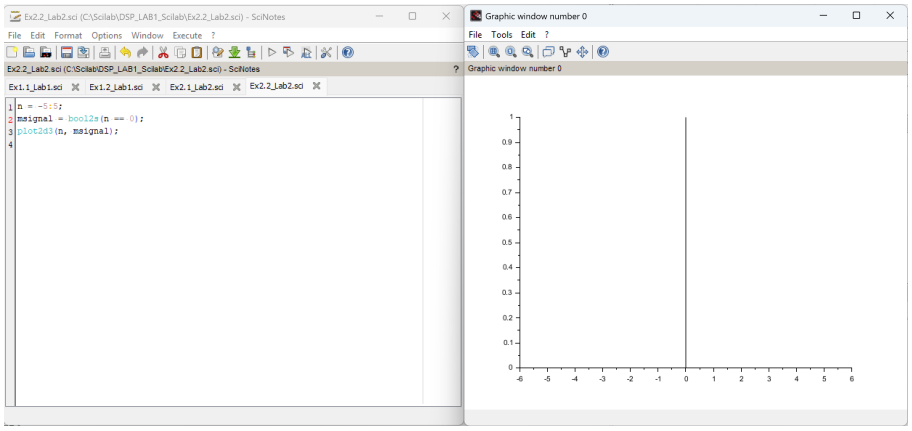
Hình 3: Đồ thị tín hiệu bước nhảy $u(n)$.

4.2.3 Bài 3:

Yêu cầu: Khảo sát mã lệnh tạo tín hiệu xung đơn vị $\delta(n)$ và giải thích kết quả.

Giải thích: Tương tự như cấu trúc của Bài tập 2, biểu thức `n == 0` kiểm tra điều kiện và chỉ trả về giá trị Đúng tại duy nhất vị trí gốc thời gian $n = 0$. Sau khi sử dụng hàm `bool2s()`, tín hiệu sinh ra chỉ có một xung duy nhất với biên độ bằng 1 tại gốc tọa độ. Điều này hoàn toàn khớp với định nghĩa toán học của tín hiệu xung đơn vị $\delta(n)$.

```
n = -5:5;
msignal = bool2s(n == 0);
plot2d3(n, msignal);
```



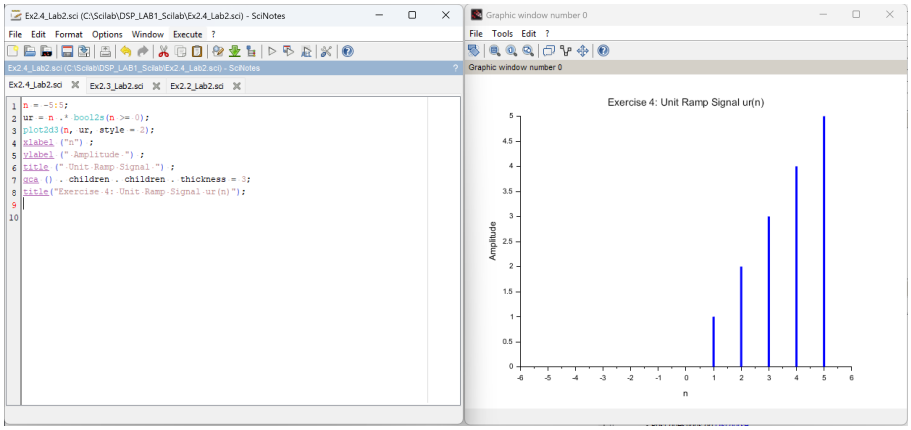
Hình 4: Đồ thị tín hiệu xung đơn vị $\delta(n)$.

4.2.4 Bài 4:

Yêu cầu: Sử dụng Scilab để vẽ tín hiệu hàm dốc $u_r(n)$ trong khoảng $n = -5:5$.

Giải thích: Tín hiệu hàm dốc được định nghĩa là $u_r(n) = n$ khi $n \geq 0$ và bằng 0 khi $n < 0$. Trong Scilab, tín hiệu này được khởi tạo bằng phép nhân từng phần tử giữa mảng thời gian `n` và mảng điều kiện bước nhảy `bool2s(n >= 0)`.

```
n = -5:5;
ur = n .* bool2s(n >= 0);
plot2d3(n, ur, style = 2);
xlabel ("n") ;
ylabel (" Amplitude ") ;
title (" Unit Ramp Signal ") ;
gca () . children . children . thickness = 3;
title("Exercise 4: Unit Ramp Signal ur(n)");
```



Hình 5: Đồ thị tín hiệu hàm dốc $u_r(n)$.

4.2.5 Bài 5:

Yêu cầu: Vẽ tín hiệu $x(n) = \{1, 3\uparrow, -2\}$ cùng với thành phần chẵn $x_e(n)$ và thành phần lẻ $x_o(n)$.

Giải thích: Dựa vào vị trí gốc $n = 0$ tại giá trị 3, ta chèn thêm các phần tử 0 (zero-padding) để đồng bộ trục thời gian đối xứng từ $n = -2$ đến $n = 2$ nhằm đảm bảo tính chính xác khi đảo trục thời gian $x(-n)$. Các thành phần được tính theo công thức lý thuyết:

$$x_e(n) = \frac{1}{2}[x(n) + x(-n)]$$
$$x_o(n) = \frac{1}{2}[x(n) - x(-n)]$$

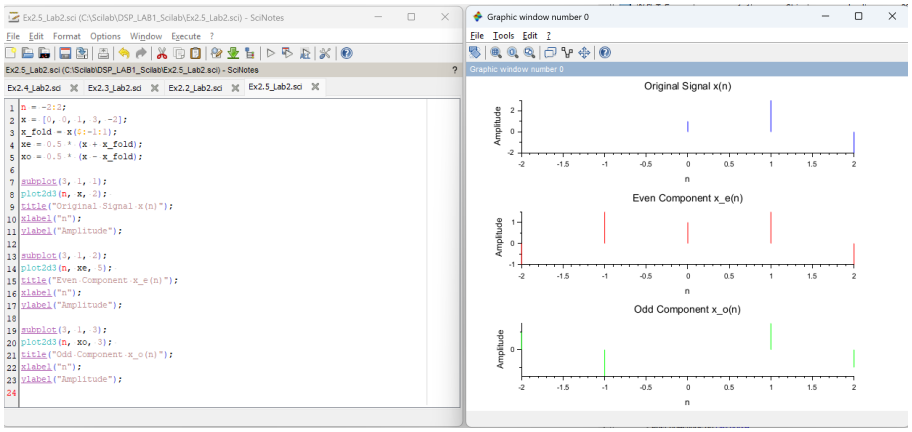
```
n = -2:2;
x = [0, 0, 1, 3, -2];
x_fold = x($:-1:1);
xe = 0.5 * (x + x_fold);
xo = 0.5 * (x - x_fold);

```

```
subplot(3, 1, 1);
plot2d3(n, x, 2);
title("Original Signal x(n)");
xlabel("n");
ylabel("Amplitude");

subplot(3, 1, 2);
plot2d3(n, xe, 5);
title("Even Component x_e(n)");
xlabel("n");
ylabel("Amplitude");

subplot(3, 1, 3);
plot2d3(n, xo, 3);
title("Odd Component x_o(n)");
xlabel("n");
ylabel("Amplitude");
```



Hình 6: Tín hiệu gốc $x(n)$ cùng các thành phần chẵn $x_e(n)$ và lẻ $x_o(n)$.

4.2.6 Bài 6:

Yêu cầu: Cho hai tín hiệu rời rạc $x_1(n) = \{0 \uparrow, 1, 3, -2\}$ và $x_2(n) = \{0, 1 \uparrow, 2, 3\}$. Tính và vẽ tín hiệu tổng $y(n) = x_1(n) + x_2(n)$.

Giải thích: Hai tín hiệu ban đầu có chiều dài và mốc thời gian khác nhau. Để thực hiện phép cộng, ta cần xác định khoảng thời gian chung bao phủ cả hai (từ $n = -1$ đến $n = 3$). Bằng kỹ thuật thêm số không (zero-padding) vào các vị trí bị khuyết, hai mảng dữ liệu được đồng bộ về cùng kích thước, sau đó ta tiến hành cộng trực tiếp từng phần tử tương ứng.

```
n = -1:3;
x1 = [0, 0, 1, 3, -2];
x2 = [0, 1, 2, 3, 0];

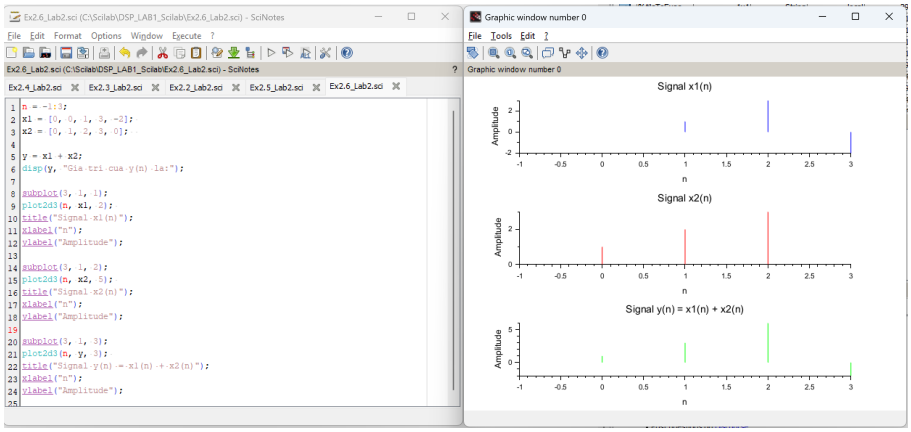
y = x1 + x2;
disp(y, "Gia tri cua y(n) la:");

subplot(3, 1, 1);
plot2d3(n, x1, 2);
title("Signal x1(n)");
```

```
xlabel("n");
ylabel("Amplitude");

subplot(3, 1, 2);
plot2d3(n, x2, 5);
title("Signal x2(n)");
xlabel("n");
ylabel("Amplitude");

subplot(3, 1, 3);
plot2d3(n, y, 3);
title("Signal y(n) = x1(n) + x2(n)");
xlabel("n");
ylabel("Amplitude");
```



Hình 7: Đồ thị biểu diễn phép cộng hai tín hiệu $y(n) = x_1(n) + x_2(n)$.

4.2.7 Bài 7:

Yêu cầu: Với cùng hai tín hiệu $x_1(n)$ và $x_2(n)$ như ở Bài tập 6, hãy tính và vẽ tín hiệu tích $y(n) = x_1(n) \cdot x_2(n)$.

Giải thích: Tương tự như phép cộng, thao tác zero-padding được sử dụng để đồng bộ trục thời gian chung từ $n = -1$ đến $n = 3$. Điểm lưu ý quan trọng nhất khi lập trình trong Scilab là phải sử dụng toán tử `.*` để thực hiện phép nhân từng phần tử (element-wise multiplication) giữa hai mảng dữ liệu rời rạc, tránh bị nhầm lẫn thành phép nhân ma trận.

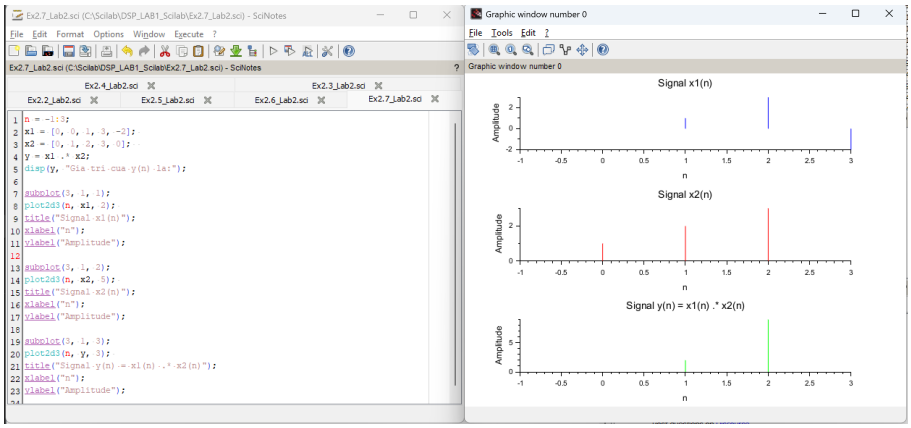
```
n = -1:3;
x1 = [0, 0, 1, 3, -2];
x2 = [0, 1, 2, 3, 0];
y = x1 .* x2;
disp(y, "Gia tri cua y(n) la:");

subplot(3, 1, 1);
plot2d3(n, x1, 2);
title("Signal x1(n)");
xlabel("n");
ylabel("Amplitude");

subplot(3, 1, 2);
```

```
plot2d3(n, x2, 5);
title("Signal x2(n)");
xlabel("n");
ylabel("Amplitude");

subplot(3, 1, 3);
plot2d3(n, y, 3);
title("Signal y(n) = x1(n) .* x2(n)");
xlabel("n");
ylabel("Amplitude");
```



Hình 8: Đồ thị biểu diễn phép nhân hai tín hiệu $y(n) = x_1(n) \cdot x_2(n)$.

4.2.8 Bài 8:

Yêu cầu: Cho tín hiệu rời rạc $x(n) = \{1, -2, 3\uparrow, 6\}$. Khảo sát và vẽ các phép biến đổi $y_1(n) = x(-n)$, $y_2(n) = x(n+3)$ và $y_3(n) = 2x(-n-2)$.

Giải thích:

- $y_1(n) = x(-n)$: Thực hiện phép đảo ngược thời gian (folding) bằng cách lật trục thời gian của tín hiệu gốc.
- $y_2(n) = x(n+3)$: Thực hiện phép dịch thời gian sang trái 3 đơn vị (advance), tín hiệu gốc xảy ra sớm hơn.
- $y_3(n) = 2x(-n-2)$: Biến đổi kết hợp bao gồm lật trục thời gian, dịch sang phải 2 đơn vị (delay) và khuếch đại biên độ lên gấp đôi.

```
nx = -2:1;
x = [1, -2, 3, 6];
ny1 = -nx($:-1:1);
y1 = x($:-1:1);

scf(1); clf(1);
subplot(2, 1, 1);
plot2d3(nx, x, 2); title("Original Signal x(n)"); xlabel("n");
    ylabel("Amplitude");
subplot(2, 1, 2);
```

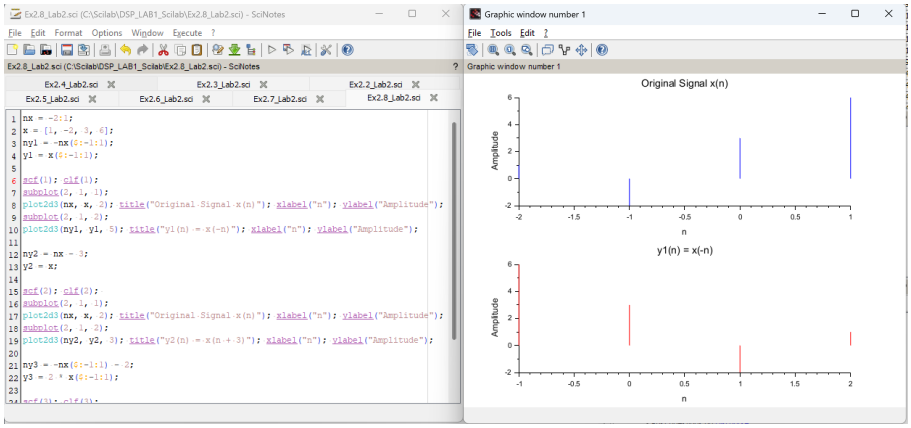
```
plot2d3(ny1, y1, 5); title("y1(n) = x(-n)"); xlabel("n"); ylabel("
    Amplitude");

ny2 = nx - 3;
y2 = x;

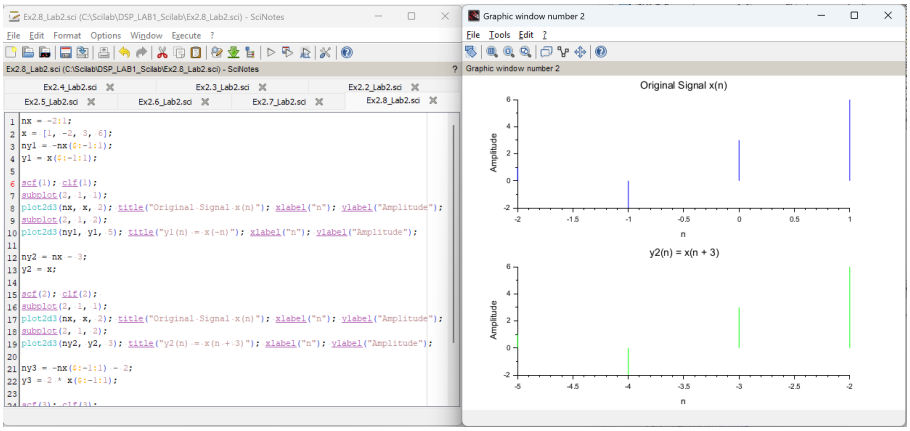
scf(2); clf(2);
subplot(2, 1, 1);
plot2d3(nx, x, 2); title("Original Signal x(n)"); xlabel("n");
    ylabel("Amplitude");
subplot(2, 1, 2);
plot2d3(ny2, y2, 3); title("y2(n) = x(n + 3)"); xlabel("n"); ylabel
    ("Amplitude");

ny3 = -nx($:-1:1) - 2;
y3 = 2 * x($:-1:1);

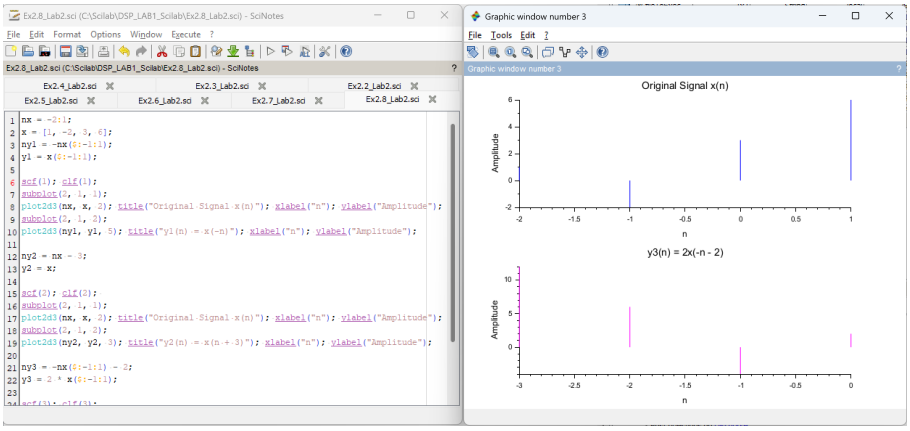
scf(3); clf(3);
subplot(2, 1, 1);
plot2d3(nx, x, 2); title("Original Signal x(n)"); xlabel("n");
    ylabel("Amplitude");
subplot(2, 1, 2);
plot2d3(ny3, y3, 6); title("y3(n) = 2x(-n - 2)"); xlabel("n");
    ylabel("Amplitude");
```



Hình 9: Đồ thị biểu diễn phép đảo thời gian $y_1(n) = x(-n)$.



Hình 10: Đồ thị biểu diễn phép dịch thời gian $y_2(n) = x(n+3)$.



Hình 11: Đồ thị biểu diễn phép biến đổi kết hợp $y_3(n) = 2x(-n-2)$.

4.2.9 Bài 1.2

Yêu cầu: Xác định các tín hiệu hình sin sau có tuần hoàn hay không và tính chu kỳ cơ bản của chúng: (a) $\cos(0.01\pi n)$, (b) $\cos\left(\pi\frac{30n}{105}\right)$, (c) $\cos(3\pi n)$, (d) $\sin(3n)$, (e) $\sin\left(\pi\frac{62n}{10}\right)$.

Lời giải: Điều kiện để tín hiệu hình sin rời rạc $\cos(2\pi f_0 n+\theta)$ hoặc $\sin(2\pi f_0 n+\theta)$ tuần hoàn là tần số chuẩn hóa f_0 phải là một số hữu tỉ ($f_0 = \frac{k}{N}$, với k, N là các số nguyên tối giản). Khi đó chu kỳ cơ bản là N.

(a) $\cos(0.01\pi n) = \cos(2\pi \cdot 0.005n)$: $f_0 = 0.005 = \frac{5}{1000} = \frac{1}{200}$ (là số hữu tỉ). Vậy tín hiệu này tuần hoàn, có chu kỳ cơ bản $N = 200$.

(b) $\cos\left(\pi\frac{30n}{105}\right) = \cos\left(2\pi\frac{15n}{105}\right) = \cos\left(2\pi\frac{n}{7}\right)$: $f_0 = \frac{1}{7}$ (là số hữu tỉ). Vậy tín hiệu này tuần hoàn, có chu kỳ cơ bản $N = 7$.

(c) $\cos(3\pi n) = \cos\left(2\pi \cdot \frac{3}{2}n\right)$: $f_0 = \frac{3}{2}$ (là số hữu tỉ). Vậy tín hiệu này tuần hoàn, có chu kỳ cơ bản $N = 2$ (vì $3\pi n = \frac{6\pi}{2}n$).

(d) $\sin(3n) = \sin\left(2\pi \cdot \frac{3}{2\pi}n\right)$: $f_0 = \frac{3}{2\pi}$ (không phải là số hữu tỉ vì chứa số π). Vậy tín hiệu này không tuần hoàn.

(e) $\sin\left(\pi\frac{62n}{10}\right) = \sin\left(2\pi\frac{31n}{10}\right)$: $f_0 = \frac{31}{10}$ (là số hữu tỉ tối giản). Vậy tín hiệu này tuần hoàn, có chu kỳ cơ bản $N = 10$.

4.2.10 Bài 1.4

Yêu cầu: (a) Chứng minh rằng chu kỳ cơ bản N_p của tập tín hiệu hàm mũ phức $s_k(n) = e^{j2\pi kn/N}$ $(k = 0, 1, 2, \dots)$ là $N_p = \frac{N}{\text{GCD}(k,N)}$, trong đó GCD là ước chung lớn nhất của k và N. (b) Tính chu kỳ cơ bản của tập này khi $N = 7$. (c) Tính chu kỳ cơ bản khi $N = 16$.

Lời giải:

(a) **Chứng minh:** Tín hiệu $s_k(n)$ tuần hoàn với chu kỳ N_p nếu $s_k(n+N_p) = s_k(n)$:

$$e^{j\frac{2\pi k(n+N_p)}{N}} = e^{j\frac{2\pi kn}{N}} \implies e^{j\frac{2\pi kN_p}{N}} = 1$$

Điều này xảy ra khi góc lũy thừa là bội số nguyên của 2π:

$$\frac{2\pi kN_p}{N} = 2\pi m \implies kN_p = mN$$

Để tìm chu kỳ cơ bản N_p nhỏ nhất, ta chia cả hai vế cho $\text{GCD}(k, N)$:

$$N_p = \frac{N}{\text{GCD}(k, N)}$$

(ĐPCM)

(b) Khi $N = 7$:

- Với $k = 0$: $N_p = \frac{7}{\text{GCD}(0,7)} = 1$.
- Với $k = 1, 2, 3, 4, 5, 6$: $\text{GCD}(k, 7) = 1$ (vì 7 là số nguyên tố). Do đó $N_p = 7$.

(c) Khi $N = 16$ (2^4):

- $k = 0 \implies N_p = 1$.
- $k = 1, 3, 5, 7, 9, 11, 13, 15$ (nguyên tố cùng nhau với 16): GCD(k, 16) = 1 $\implies N_p = 16$.
- $k = 2, 6, 10, 14$ (GCD = 2) $\implies N_p = \frac{16}{2} = 8$.
- $k = 4, 12$ (GCD = 4) $\implies N_p = \frac{16}{4} = 4$.
- $k = 8$ (GCD = 8) $\implies N_p = \frac{16}{8} = 2$.

4.2.11 Bài 1.7

Yêu cầu: Một tín hiệu analog chứa các tần số lên đến 10 kHz. (a) Dải tần số lấy mẫu nào cho phép khôi phục chính xác tín hiệu này từ các mẫu của nó? (b) Giả sử lấy mẫu với tần số $F_s = 8$ kHz. Hãy phân tích điều gì xảy ra với tần số $F_1 = 5$ kHz. (c) Lặp lại phần (b) cho tần số $F_2 = 9$ kHz.

Lời giải:

(a) Dải tần số lấy mẫu: Theo định lý lấy mẫu (Nyquist), tần số lấy mẫu phải thỏa mãn $F_s \geq 2F_{\max}$. Với $F_{\max} = 10$ kHz, ta có $F_s \geq 20$ kHz.

(b) Với $F_s = 8$ kHz và $F_1 = 5$ kHz: Tần số gấp nếp (folding frequency) là $\frac{F_s}{2} = 4$ kHz. Vì $F_1 = 5$ kHz > 4 kHz, hiện tượng chồng phổ (aliasing) xảy ra. Tần số xuất hiện sau khi lấy mẫu là:

$$F_{1,\text{alias}} = |F_1 - F_s| = |5 - 8| = 3 \text{ kHz}$$

(c) Với $F_s = 8$ kHz và $F_2 = 9$ kHz: Tần số gấp nếp là 4 kHz. Hiện tượng chồng phổ xảy ra:

$$F_{2,\text{alias}} = |9 - 8| = 1 \text{ kHz}$$

4.2.12 Bài 1.11

Yêu cầu: Cho hệ thống xử lý tín hiệu với chu kỳ lấy mẫu của bộ A/D là $T = 5$ ms và bộ D/A là $T' = 1$ ms. Đầu vào là $x_a(t) = 3\cos(100\pi t) + 2\sin(250\pi t)$ (với t tính bằng giây). Bộ lọc sau (postfilter) loại bỏ mọi thành phần tần số lớn hơn $F_s/2$. Hãy xác định tín hiệu đầu ra $y_a(t)$.

Lời giải:

Bước 1: Quá trình lấy mẫu qua bộ A/D

Tần số lấy mẫu: $F_s = \frac{1}{T} = \frac{1}{0.005} = 200$ Hz.

Tín hiệu rời rạc $x(n)$ được thu được bằng cách thay $t = nT = 0.005n = \frac{n}{200}$ vào $x_a(t)$:

$$x(n) = x_a(nT) = 3\cos\left(100\pi \cdot \frac{n}{200}\right) + 2\sin\left(250\pi \cdot \frac{n}{200}\right)$$

$$= 3\cos\left(\frac{100\pi n}{200}\right) + 2\sin\left(\frac{250\pi n}{200}\right)$$

$$= 3\cos\left(\frac{\pi n}{2}\right) + 2\sin\left(\frac{5\pi n}{4}\right)$$

Bước 2: Biến đổi lượng giác thu gọn

Sử dụng tính chất của hàm sin: $\sin\left(\frac{5\pi n}{4}\right) = \sin\left(2\pi n - \frac{3\pi n}{4}\right) = -\sin\left(\frac{3\pi n}{4}\right)$.

Biểu thức tín hiệu rời rạc sau khi rút gọn:

$$x(n) = 3\cos\left(\frac{\pi n}{2}\right) - 2\sin\left(\frac{3\pi n}{4}\right)$$

Bước 3: Quá trình tái tạo qua bộ D/A và qua bộ lọc

Bộ D/A chuyển đổi tín hiệu rời rạc sang tín hiệu liên tục với chu kỳ $T' = 1$ ms $= \frac{1}{1000}$ s, bằng cách thay $n = \frac{t}{T'} = 1000t$.

$$y_a(t) = 3\cos\left(\frac{\pi \cdot 1000t}{2}\right) - 2\sin\left(\frac{3\pi \cdot 1000t}{4}\right)$$

$$= 3\cos(500\pi t) - 2\sin(750\pi t)$$

4.2.13 Bài 1.15

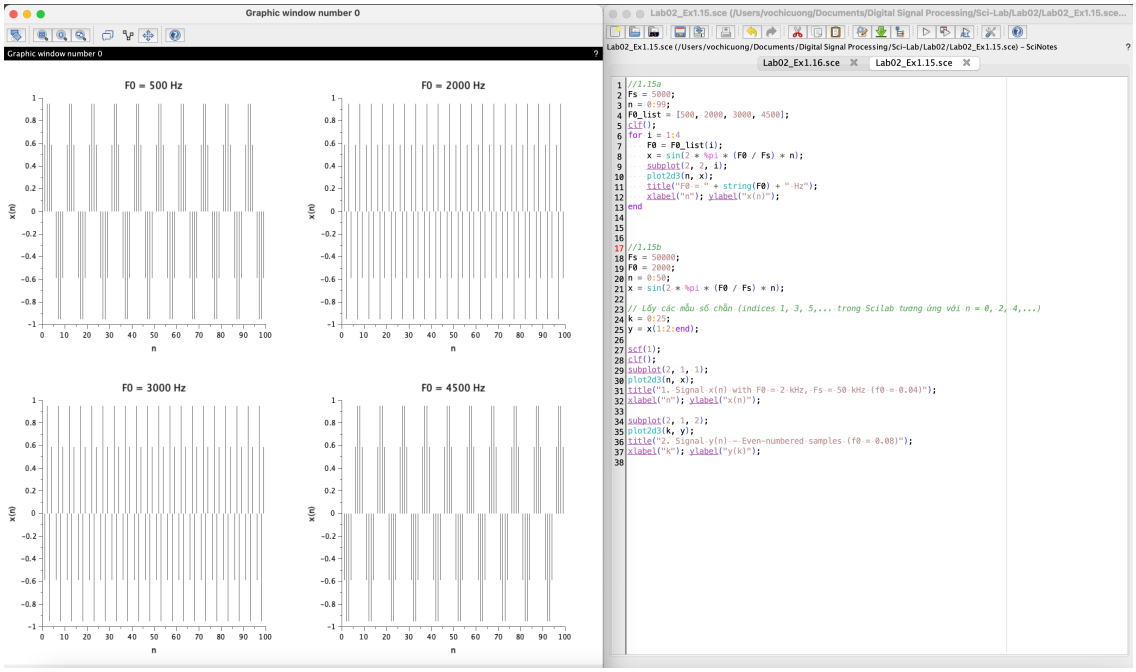
(a) Yêu cầu: Vẽ tín hiệu $x(n)$ với $0 \leq n \leq 99$ khi $F_s = 5$ kHz và các tần số $F_0 = 0.5, 2, 3,$ và 4.5 kHz. Giải thích sự giống và khác nhau giữa các đồ thị.

Giải thích lý thuyết & kết quả: Tần số gấp nếp (folding frequency) là $\frac{F_s}{2} = \frac{5}{2} = 2.5$ kHz.

- **Cặp 1 (0.5 kHz và 4.5 kHz):** $F_0 = 0.5$ kHz nằm trong dải cơ bản. $F_0 = 4.5$ kHz vượt quá 2.5 kHz, tạo ra hiện tượng chồng phổ (aliasing) với tần số giả lập là $|4.5 - 5| = 0.5$ kHz. Do đó, đồ thị của 0.5 kHz và 4.5 kHz giống hệt nhau (chu kỳ 10 mẫu).

- **Cặp 2 (2 kHz và 3 kHz):** $F_0 = 2$ kHz nằm trong dải cơ bản. $F_0 = 3$ kHz bị chồng phổ về tần số giả lập $|3 - 5| = 2$ kHz. Do đó, đồ thị của 2 kHz và 3 kHz giống hệt nhau (chu kỳ 2.5 mẫu hoặc dạng sóng lặp lại tương ứng).

```
Fs = 5000;
n = 0:99;
F0_list = [500, 2000, 3000, 4500];
clf();
for i = 1:4
    F0 = F0_list(i);
    x = sin(2 * %pi * (F0 / Fs) * n);
    subplot(2, 2, i);
    plot2d3(n, x);
    title("F0 = " + string(F0) + " Hz");
    xlabel("n"); ylabel("x(n)");
end
```



(b) Yêu cầu: Cho $F_0 = 2$ kHz và $F_s = 50$ kHz.

1. Vẽ tín hiệu $x(n)$. Tần số f_0 của tín hiệu $x(n)$ là bao nhiêu?

2. Vẽ tín hiệu $y(n)$ được tạo ra bằng cách lấy các mẫu có chỉ số chẵn của $x(n)$. Đây có phải là tín hiệu hình sin không? Tại sao? Nếu có, tần số của nó là bao nhiêu?

   Lời giải chi tiết:

- **Tần số f_0 của $x(n)$:** Công thức tín hiệu rời rạc: $x(n) = \sin\left(2\pi\frac{F_0}{F_s}n\right)$. Thay số:

$$x(n) = \sin\left(2\pi\frac{2}{50}n\right) = \sin\left(2\pi \cdot 0.04n\right)$$

  Vậy tần số chuẩn hóa (normalized frequency) của $x(n)$ là: $f_0 = 0.04$ chu kỳ/mẫu (hoặc tần số góc $\omega_0 = 0.08\pi$ rad/mẫu).

- **Khảo sát tín hiệu $y(n)$ (lấy các mẫu số chẵn):** Các mẫu chẵn của $x(n)$ có dạng $n = 2k$ (với k là số nguyên). Biểu thức của:

$$y(k) = x(2k) = \sin\left(2\pi \cdot \frac{2}{50} \cdot (2k)\right) = \sin\left(2\pi \cdot \frac{4}{50}k\right) = \sin\left(2\pi \cdot 0.08k\right)$$

  Kết luận: $y(n)$ hoàn toàn là một tín hiệu hình sin. Lý do là vì việc chọn các mẫu cách đều (giảm mẫu đi một nửa) vẫn giữ nguyên dạng hàm điều hòa (hình sin) nhưng làm tăng tần số chuẩn hóa gấp đôi.

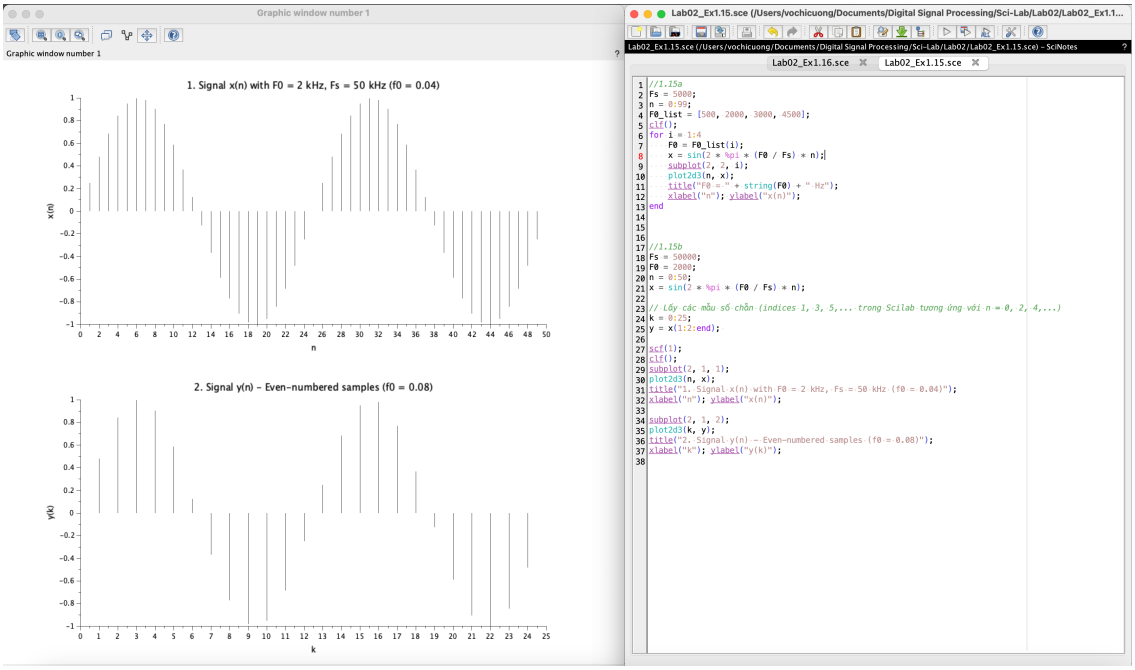
  Tần số mới của tín hiệu $y(n)$ là: $f_{0,\text{new}} = 0.08$ chu kỳ/mẫu.

```
Fs = 50000;
F0 = 2000;
n = 0:50;
x = sin(2 * %pi * (F0 / Fs) * n);

k = 0:25;
y = x(1:2:end);

scf(1);
clf();
subplot(2, 1, 1);
plot2d3(n, x);
title("1. Signal x(n) with F0 = 2 kHz, Fs = 50 kHz (f0 = 0.04)");
xlabel("n"); ylabel("x(n)");

subplot(2, 1, 2);
plot2d3(k, y);
title("2. Signal y(n) - Even-numbered samples (f0 = 0.08)");
xlabel("k"); ylabel("y(k)");
```



4.2.14 Bài 1.16

(a) **Lượng tử hóa bằng phương pháp cắt cụt (Truncation):**
Yêu cầu: Viết chương trình với $f_0 = 1/50$ và $N = 200$. Lượng tử hóa tín hiệu $x(n) = \sin(2\pi f_0 n)$ với các mức lượng tử hóa $L = 64, 128, 256$. Với mỗi trường hợp, vẽ các tín hiệu $x(n)$, tín hiệu lượng tử $x_q(n)$, sai số $e(n)$ và tính giá trị SQNR tương ứng.

(b) **Lượng tử hóa bằng phương pháp làm tròn (Rounding):**
Yêu cầu: Lặp lại toàn bộ các bước ở phần (a) nhưng thay phương pháp cắt cụt bằng phương pháp làm tròn.

(c) **Nhận xét kết quả giữa phần (a) và (b):**

- **Cắt cụt (Truncation):** Sai số lượng tử hóa có xu hướng dịch lệch về một phía (thường nằm trong khoảng $[-\Delta, 0]$ hoặc $[0, \Delta]$ tùy theo cách định nghĩa) khiến sai số trung bình khác không.
- **Làm tròn (Rounding):** Sai số lượng tử hóa dao động đối xứng quanh điểm 0 (trong khoảng $\left[-\frac{\Delta}{2}, \frac{\Delta}{2}\right]$), do đó giảm thiểu được công suất nhiễu tốt hơn so với phương pháp cắt cụt.

(d) **So sánh SQNR thực nghiệm với lý thuyết:**
Công thức lý thuyết (1.4.32): SQNR (dB) $= 1.76 + 6.02b$, trong đó b là số bit tương ứng với số mức lượng tử hóa ($L = 2^b \implies b = \log_2 L$).

- Với $L = 64 \implies b = 6$ bit $\implies$ SQNR ≈ 37.88 dB.
- Với $L = 128 \implies b = 7$ bit $\implies$ SQNR ≈ 43.90 dB.
- Với $L = 256 \implies b = 8$ bit $\implies$ SQNR ≈ 49.92 dB.

Nhận xét: Giá trị SQNR tính toán từ thực nghiệm (qua mô phỏng) rất gần với giá trị lý thuyết dự đoán. Cứ tăng thêm 1 bit (tương đương gấp đôi số mức lượng tử), giá trị SQNR thực tế tăng lên khoảng 6 dB, hoàn toàn phù hợp với lý thuyết.

```
f0 = 1 / 50;
N = 200;
n = 0:(N-1);
x = sin(2 * %pi * f0 * n);

Px = sum(x.^2) / N;

levels = [64, 128, 256];

disp("--- KET QUA MO PHONG SQNR ---");

for i = 1:length(levels)
    L = levels(i);
    b = log2(L);
    sqnr_theory = 1.76 + 6.02 * b;

    xmin = -1; xmax = 1;
    delta = (xmax - xmin) / L;

    xq_trunc = floor((x - xmin) / delta) * delta + xmin;
    eq_trunc = xq_trunc - x;
    Pq_trunc = sum(eq_trunc.^2) / N;
    sqnr_trunc = 10 * log10(Px / Pq_trunc);

    xq_round = round((x - xmin) / delta) * delta + xmin;
    eq_round = xq_round - x;
    Pq_round = sum(eq_round.^2) / N;
    sqnr_round = 10 * log10(Px / Pq_round);

    mprintf("\nSo muc L = %d (b = %d bits):", L, b);
    mprintf("\n  + SQNR Ly thuyet: %.2f dB", sqnr_theory);
    mprintf("\n  + SQNR Thuc nghiem (Truncation): %.2f dB",
   sqnr_trunc);
    mprintf("\n  + SQNR Thuc nghiem (Rounding):    %.2f dB",
   sqnr_round);
end

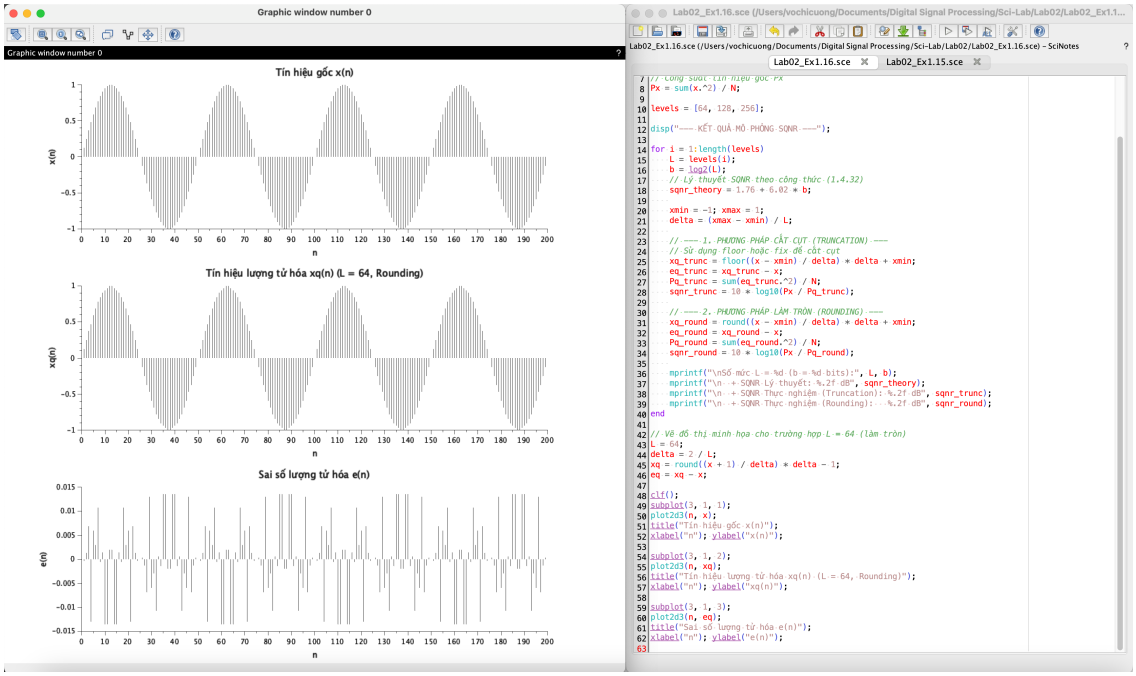
L = 64;
delta = 2 / L;
xq = round((x + 1) / delta) * delta - 1;
eq = xq - x;

clf();
subplot(3, 1, 1);
plot2d3(n, x);
title("Tin hieu goc x(n)");
xlabel("n"); ylabel("x(n)");

subplot(3, 1, 2);
plot2d3(n, xq);
title("Tin hieu luong tu hoa xq(n) (L = 64, Rounding)");
xlabel("n"); ylabel("xq(n)");

```

```
subplot(3, 1, 3);
plot2d3(n, eq);
title("Sai so luong tu hoa e(n)");
xlabel("n"); ylabel("e(n)");
```




```
  "--- KẾT QUẢ MÔ PHỎNG SQNR ---"

Số mức L = 64 (b = 6 bits):
  + SQNR Lý thuyết: 37.88 dB
  + SQNR Thực nghiệm (Truncation): 31.33 dB
  + SQNR Thực nghiệm (Rounding):   39.20 dB
Số mức L = 128 (b = 7 bits):
  + SQNR Lý thuyết: 43.90 dB
  + SQNR Thực nghiệm (Truncation): 37.36 dB
  + SQNR Thực nghiệm (Rounding):   46.53 dB
Số mức L = 256 (b = 8 bits):
  + SQNR Lý thuyết: 49.92 dB
  + SQNR Thực nghiệm (Truncation): 43.77 dB
  + SQNR Thực nghiệm (Rounding):   51.41 dB
```

5 Bàn luận

Thông qua các bài tập thực hành của Lab 1&2, sinh viên đã nắm bắt được cú pháp cơ bản của Scilab, rút ra được một số kinh nghiệm và lưu ý quan trọng khi biểu diễn và xử lý tín hiệu rời rạc bằng phần mềm Scilab:

- **Quản lý trục thời gian (Time Vector):** Khác với lý thuyết toán học nơi chỉ số n có thể nhận giá trị âm, chỉ số mảng (index) trong Scilab mặc định luôn bắt đầu từ 1. Do đó, để biểu diễn đúng tín hiệu, ta luôn phải định nghĩa một vector trục thời gian riêng biệt (ví dụ `n = -5:5`) song song với vector biên độ, giúp xác định chính xác vị trí của gốc tọa độ $n = 0$.
- **Kỹ thuật chèn số không (Zero-padding):** Khi thực hiện các phép toán trên hai tín hiệu khác nhau về chiều dài và mốc thời gian (như phép cộng ở Bài 6, phép nhân ở Bài 7), hoặc khi cần tính thành phần chẵn/lẻ yêu cầu trục thời gian đối xứng (Bài 5), việc chèn thêm các giá trị 0 (zero-padding) để đồng bộ hóa kích thước mảng là bước tiền xử lý bắt buộc. Nếu bỏ qua bước này, Scilab sẽ báo lỗi không tương thích kích thước mảng.
- **Phép toán trên từng phần tử (Element-wise operations):** Cần đặc biệt phân biệt giữa phép nhân ma trận và phép nhân từng phần tử. Khi nhân hai tín hiệu rời rạc, bắt buộc phải sử dụng toán tử `.*` thay vì dấu `*` thông thường. Tương tự đối với các điều kiện logic để tạo tín hiệu cơ bản, hàm `bool2s()` cực kỳ hữu ích để chuyển đổi mảng True/False về dạng số học 1 và 0.
- **Biến đổi tín hiệu linh hoạt:** Cú pháp lập trình mảng trong Scilab cung cấp các công cụ rất mạnh mẽ để biến đổi tín hiệu. Chẳng hạn, để lật trục thời gian (folding), ta chỉ cần dùng cú pháp truy xuất ngược `($:-1:1)`. Để dịch thời gian (shifting), thay vì can thiệp vào biên độ, ta chỉ cần cộng hoặc trừ số nguyên trực tiếp trên mảng vector thời gian một cách nhanh chóng.

6 Tài Liệu Tham Khảo

1. Tài liệu hướng dẫn Scilab cơ bản:
https://www.scilab.org/sites/default/files/Scilab_beginners.pdf
2. Mã nguồn thực hành Lab 1&2 (Github):
https://github.com/neit087/DSP_Scilab.git